Evaluating Matrix Tiling Multiplication For Asymmetric Data Cost

George Ginzburg, Areg Mkhitaryan

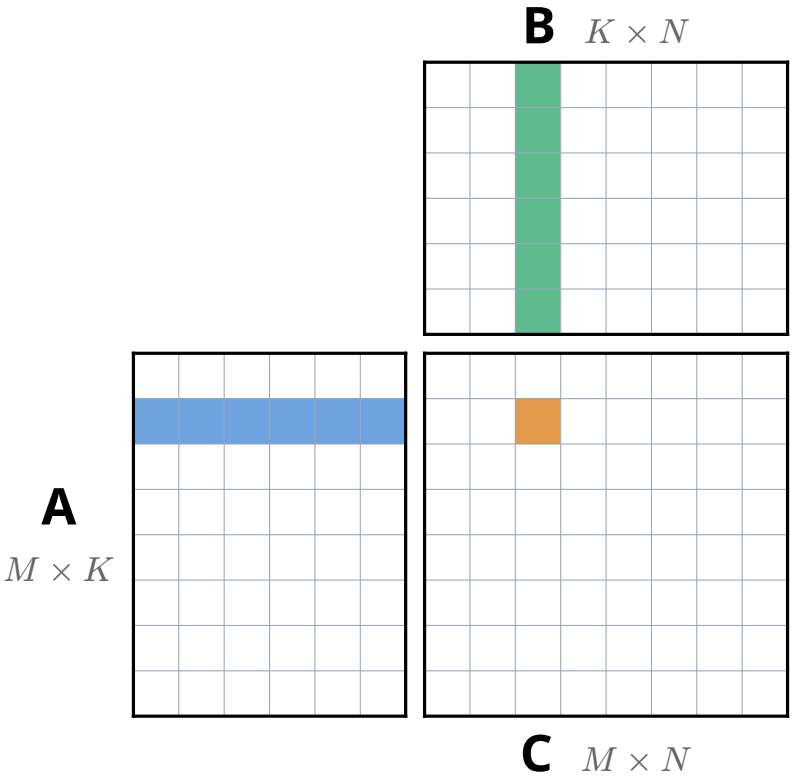
Project Aleph Spring 2026 | November 08, 2026

Motivation: Random Matrices In Practice

- **scikit-learn** random projections: data $\times$ a random Gaussian or sparse matrix
- **scikit-learn** PCA: randomized SVD, the default for large inputs
- **PyTorch** `torch.svd_lowrank`: the same randomized SVD
- **SciPy** `clarkson_woodruff_transform`: a random sketch for least squares
- **scikit-learn** `RBFSampler`: random features for kernel methods
- **Apache Spark**: random projections to find similar items

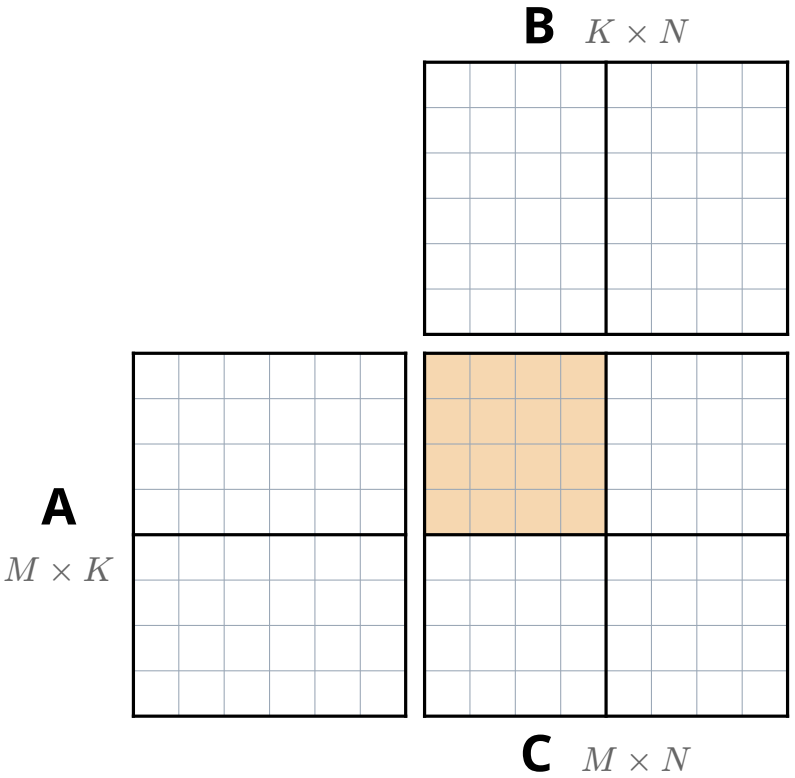
How Tiling Works

- **One element of C = a row of A × a column of B**
- Tiling: compute C one tile ($T_M \times T_N$) at a time
- A tile needs a strip of A and a panel of B
- Step k: a column of A × a row of B updates the whole tile
- Next step
- … until the last step, the tile is done
- Next tile: the **same** strip of A, a new panel of B
- Next row of tiles: a new strip of A, B again from the left



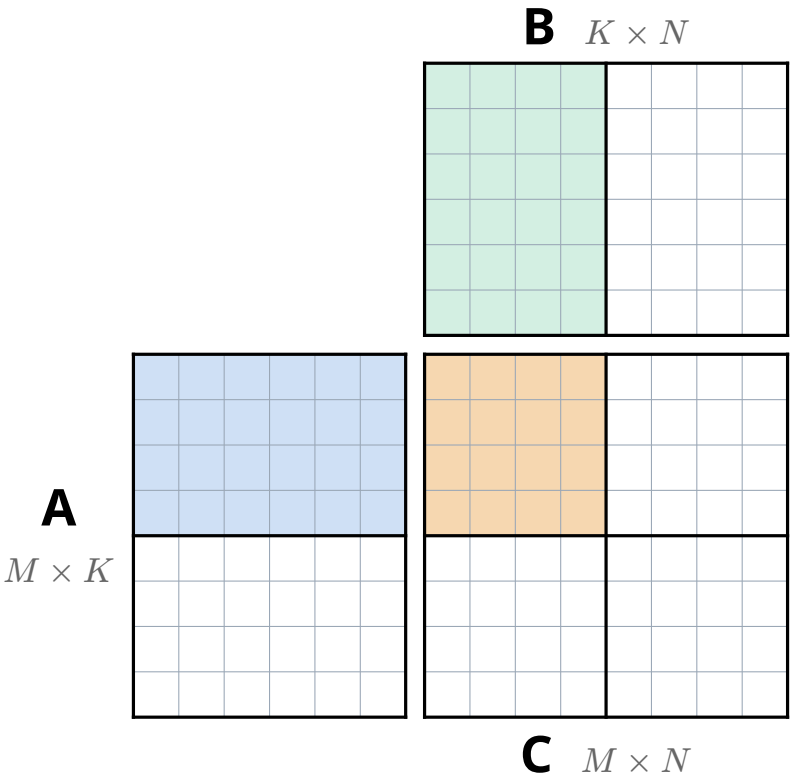
How Tiling Works

- One element of C = a row of A × a column of B
- **Tiling: compute C one tile ($T_M \times T_N$) at a time**
- A tile needs a strip of A and a panel of B
- Step k: a column of A × a row of B updates the whole tile
- Next step
- … until the last step, the tile is done
- Next tile: the **same** strip of A, a new panel of B
- Next row of tiles: a new strip of A, B again from the left



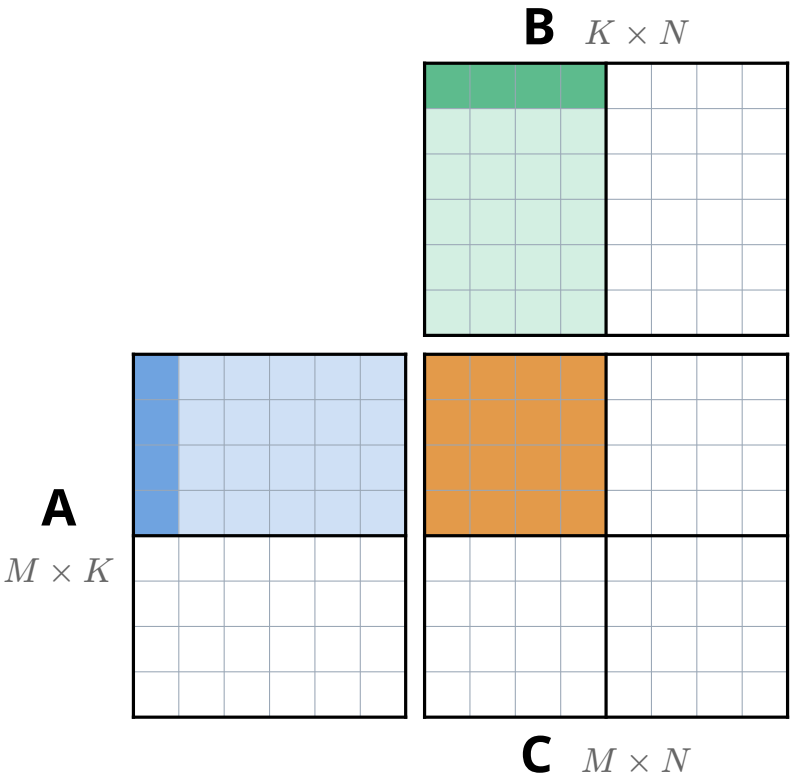
How Tiling Works

- One element of C = a row of A × a column of B
- Tiling: compute C one tile ($T_M \times T_N$) at a time
- **A tile needs a strip of A and a panel of B**
- Step k: a column of A × a row of B updates the whole tile
- Next step
- … until the last step, the tile is done
- Next tile: the **same** strip of A, a new panel of B
- Next row of tiles: a new strip of A, B again from the left



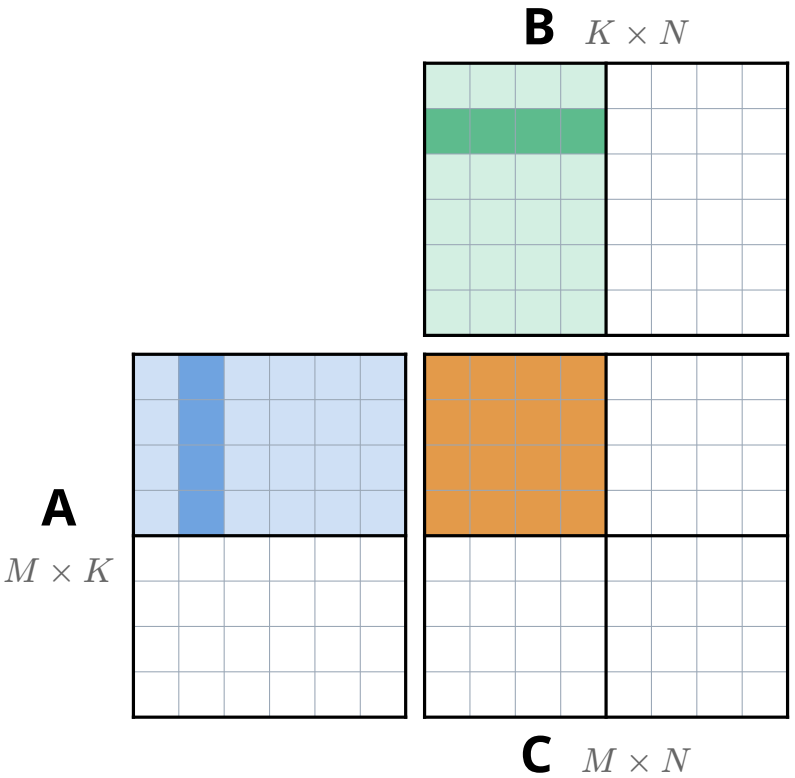
How Tiling Works

- One element of C = a row of A × a column of B
- Tiling: compute C one tile ($T_M \times T_N$) at a time
- A tile needs a strip of A and a panel of B
- **Step k: a column of A × a row of B updates the whole tile**
- Next step
- … until the last step, the tile is done
- Next tile: the **same** strip of A, a new panel of B
- Next row of tiles: a new strip of A, B again from the left



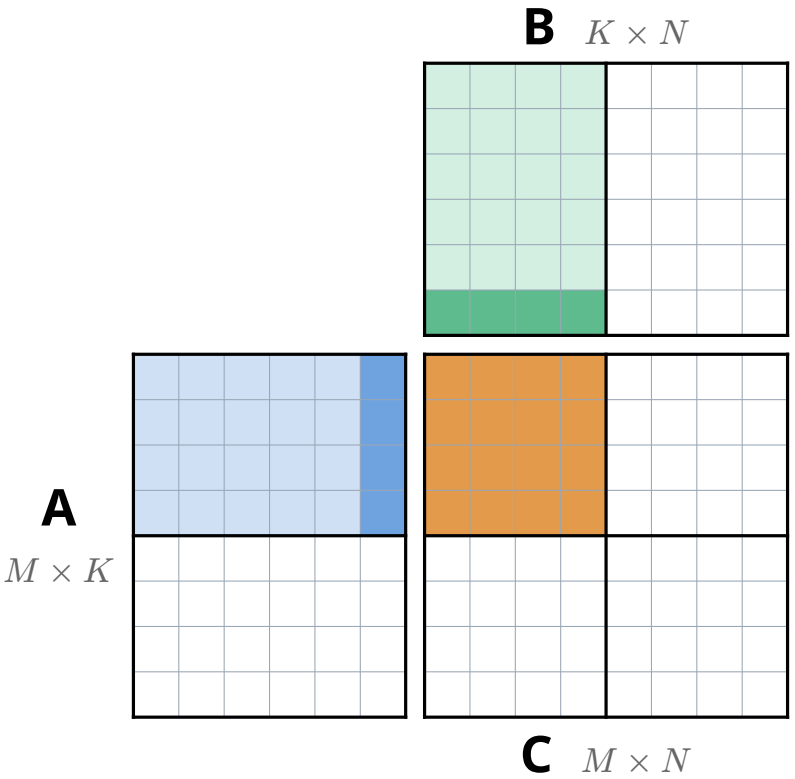
How Tiling Works

- One element of C = a row of A × a column of B
- Tiling: compute C one tile ($T_M \times T_N$) at a time
- A tile needs a strip of A and a panel of B
- Step k: a column of A × a row of B updates the whole tile
- **Next step**
- … until the last step, the tile is done
- Next tile: the **same** strip of A, a new panel of B
- Next row of tiles: a new strip of A, B again from the left



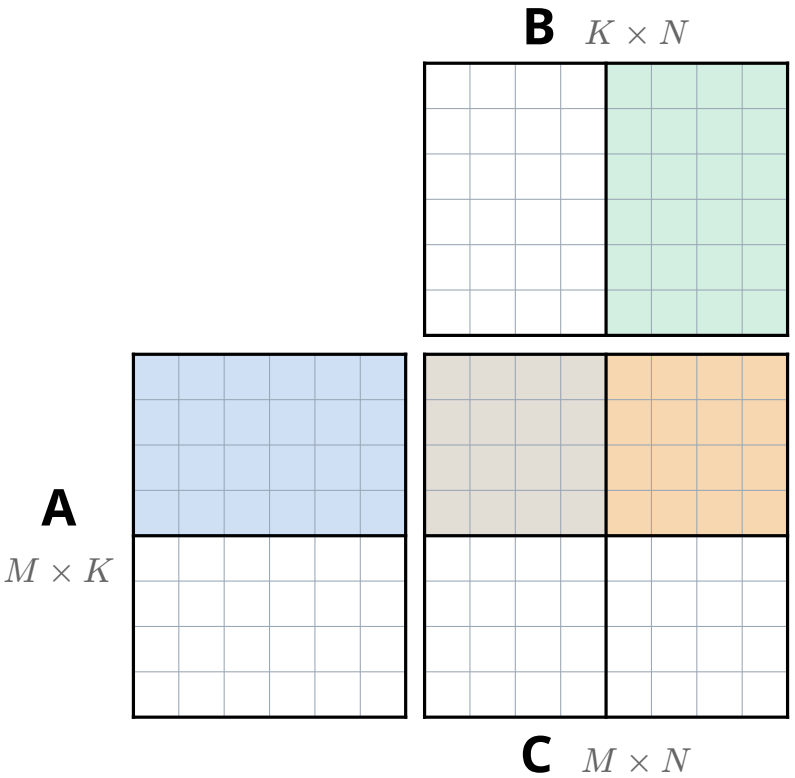
How Tiling Works

- One element of C = a row of A × a column of B
- Tiling: compute C one tile ($T_M \times T_N$) at a time
- A tile needs a strip of A and a panel of B
- Step k: a column of A × a row of B updates the whole tile
- Next step
- **… until the last step, the tile is done**
- Next tile: the **same** strip of A, a new panel of B
- Next row of tiles: a new strip of A, B again from the left



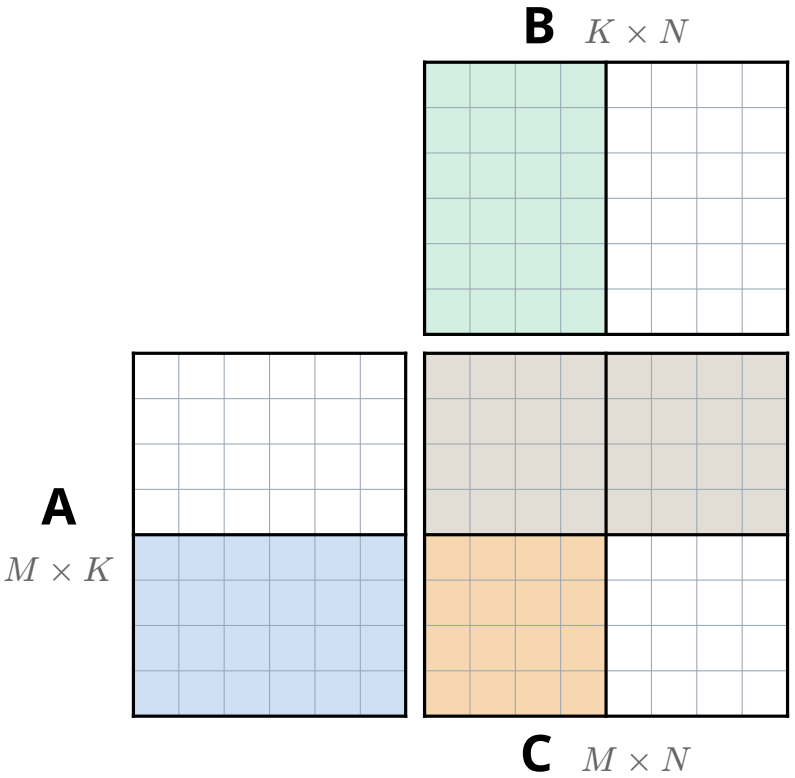
How Tiling Works

- One element of C = a row of A × a column of B
- Tiling: compute C one tile ($T_M \times T_N$) at a time
- A tile needs a strip of A and a panel of B
- Step k: a column of A × a row of B updates the whole tile
- Next step
- … until the last step, the tile is done
- **Next tile: the same strip of A, a new panel of B**
- Next row of tiles: a new strip of A, B again from the left



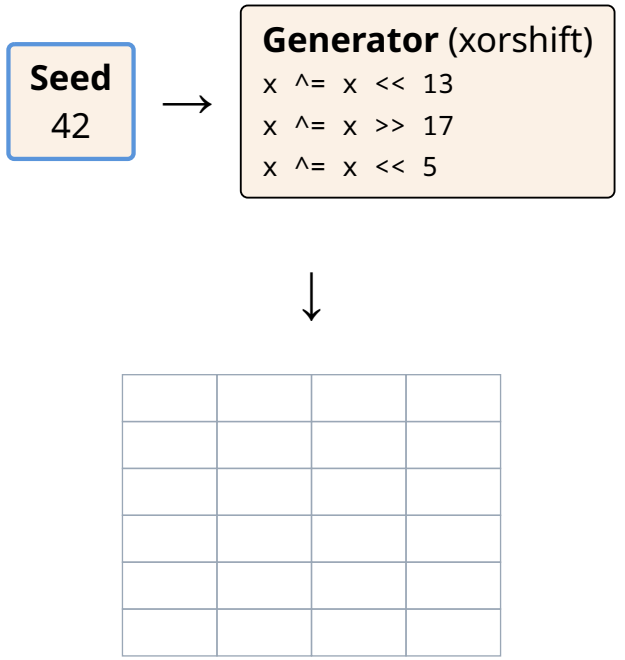
How Tiling Works

- One element of C = a row of A × a column of B
- Tiling: compute C one tile ($T_M \times T_N$) at a time
- A tile needs a strip of A and a panel of B
- Step k: a column of A × a row of B updates the whole tile
- Next step
- … until the last step, the tile is done
- Next tile: the **same** strip of A, a new panel of B
- **Next row of tiles: a new strip of A, B again from the left**



How A PRNG Works

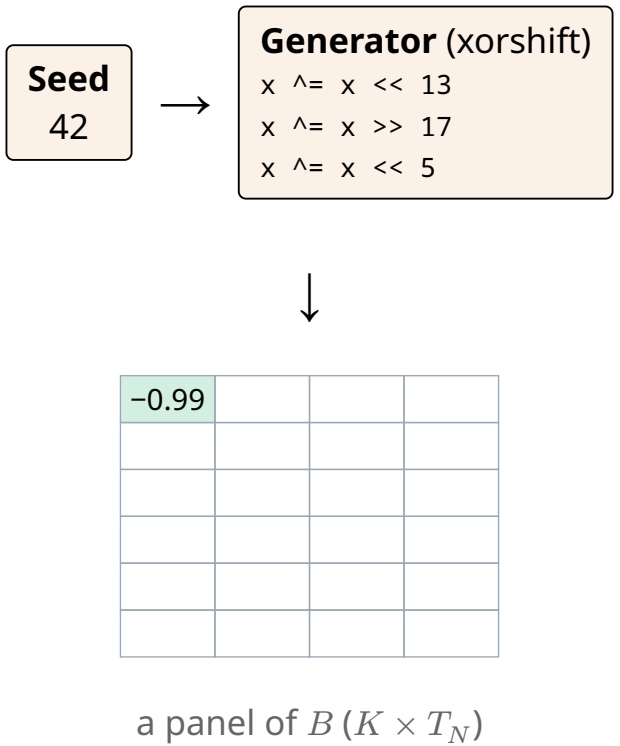
- **Load a seed**
- Each step updates the state and gives one number of B, g_c cycles
- The numbers fill B in the order the core uses it
- The whole panel of B, nothing read from memory
- Same seed $\Rightarrow$ same numbers: B is made again for every row of tiles



a panel of B ($K \times T_N$)

How A PRNG Works

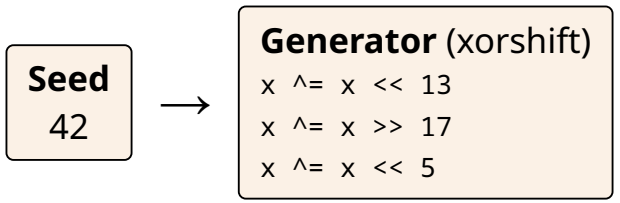
- Load a seed
- **Each step updates the state and gives one number of B, g_c cycles**
- The numbers fill B in the order the core uses it
- The whole panel of B, nothing read from memory
- Same seed $\Rightarrow$ same numbers: B is made again for every row of tiles



a panel of B ($K \times T_N$)

How A PRNG Works

- Load a seed
- Each step updates the state and gives one number of B, g_c cycles
- **The numbers fill B in the order the core uses it**
- The whole panel of B, nothing read from memory
- Same seed $\Rightarrow$ same numbers: B is made again for every row of tiles

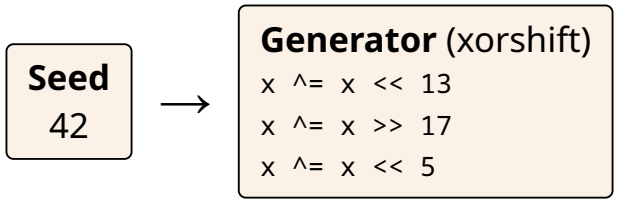


−0.99	0.32	−0.78	0.7
0.75	−0.33	0.73	0.13

a panel of B ($K \times T_N$)

How A PRNG Works

- Load a seed
- Each step updates the state and gives one number of B, g_c cycles
- The numbers fill B in the order the core uses it
- **The whole panel of B, nothing read from memory**
- Same seed $\Rightarrow$ same numbers: B is made again for every row of tiles

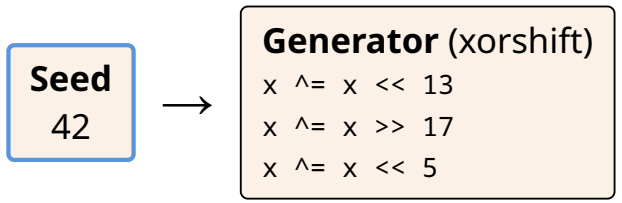


−0.99	0.32	−0.78	0.7
0.75	−0.33	0.73	0.13
0.45	−0.5	0.39	0.57
0.42	−0.48	−0.09	−0.49
−0.75	−0.98	0.62	0.26
0.51	0.13	0.12	0.14

a panel of B ($K \times T_N$)

How A PRNG Works

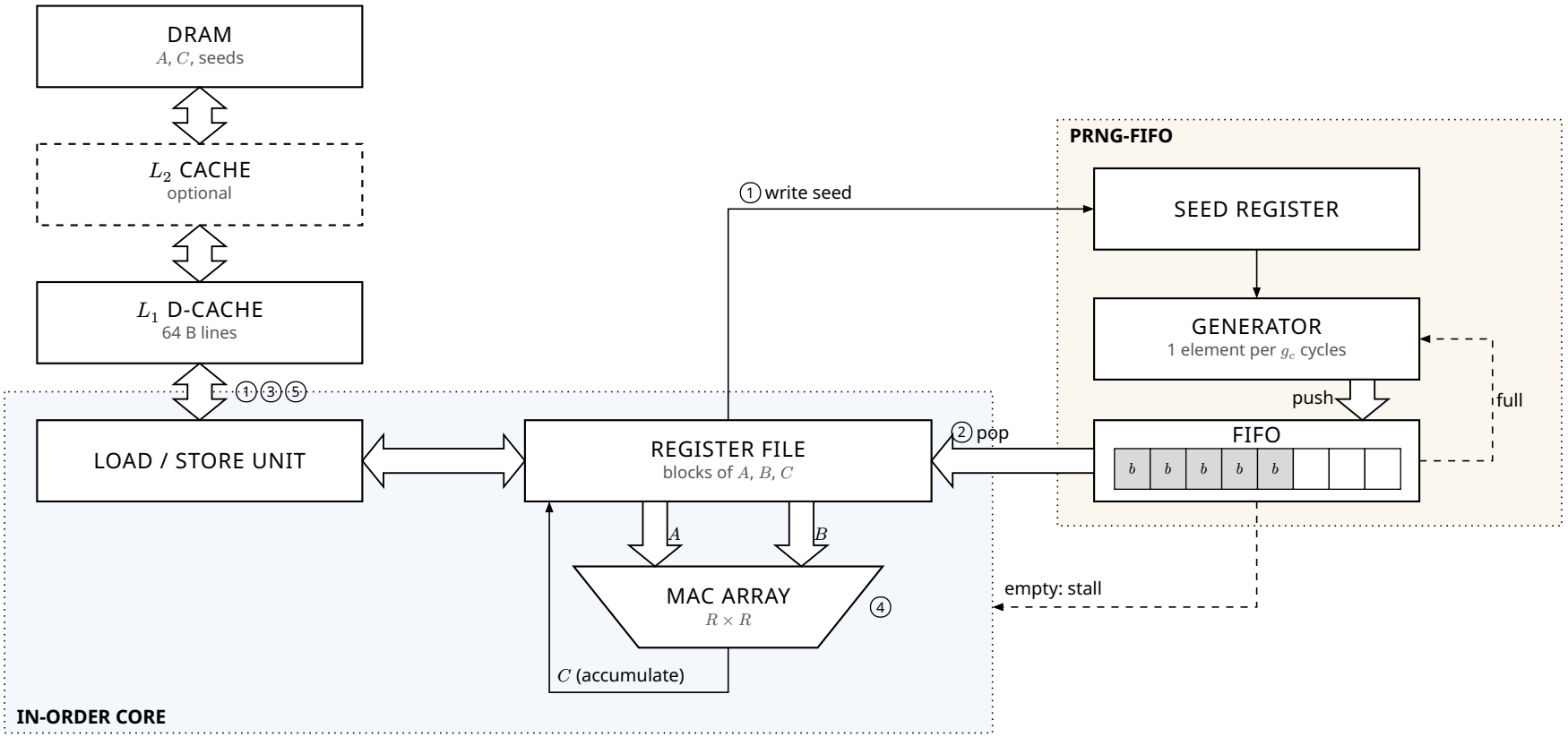
- Load a seed
- Each step updates the state and gives one number of B, g_c cycles
- The numbers fill B in the order the core uses it
- The whole panel of B, nothing read from memory
- **Same seed $\Rightarrow$ same numbers: B is made again for every row of tiles**



−0.99	0.32	−0.78	0.7
0.75	−0.33	0.73	0.13
0.45	−0.5	0.39	0.57
0.42	−0.48	−0.09	−0.49
−0.75	−0.98	0.62	0.26
0.51	0.13	0.12	0.14

a panel of B ($K \times T_N$)
again, the same numbers

The Hardware System

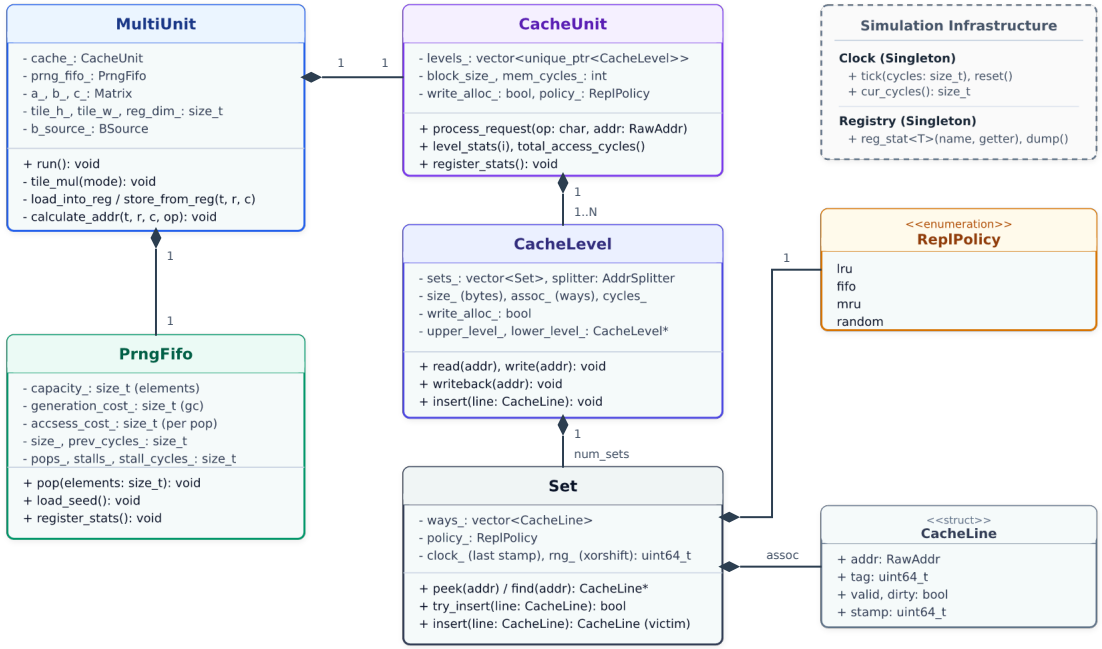


① seed ② pop B (wait if empty) ③ load A, C ④ multiply ⑤ store C

In-order core, only the generator runs in parallel

The asymm_tiling Simulator

- Cycle accurate cache simulator built in C++20
- Correctness:
  - diff-tested against a Python reference cache
  - Hypothesis traces, invariants



Default Parameters

Matrices $M \times K \times N$	$192 \times 256 \times 256$, 4-byte elements
Register block	4×4
L_1	16 KB, 64-byte lines, fully associative, LRU, 4 cycles
Memory latency	180 cycles
Generation cost g_c	swept, 0 to 600 cycles per element
FIFO	2^{14} elements, 2 cycles per pop
Compute unit	free

Every marker in a plot is a simulated point

Research Goals

1. Build a cycle-level simulator, and check it against the Bharadwaj results
2. Find what the best tile looks like, and how it changes with g_c
3. Build a model that predicts the best tile
4. Check how well the model chooses the tile, also for other matrices
5. Answer design questions for engineers who use or build such a system
6. Check the results on a more realistic core with gem5

01

Experiments And Results

Previous Results By Bharadwaj And Demmel

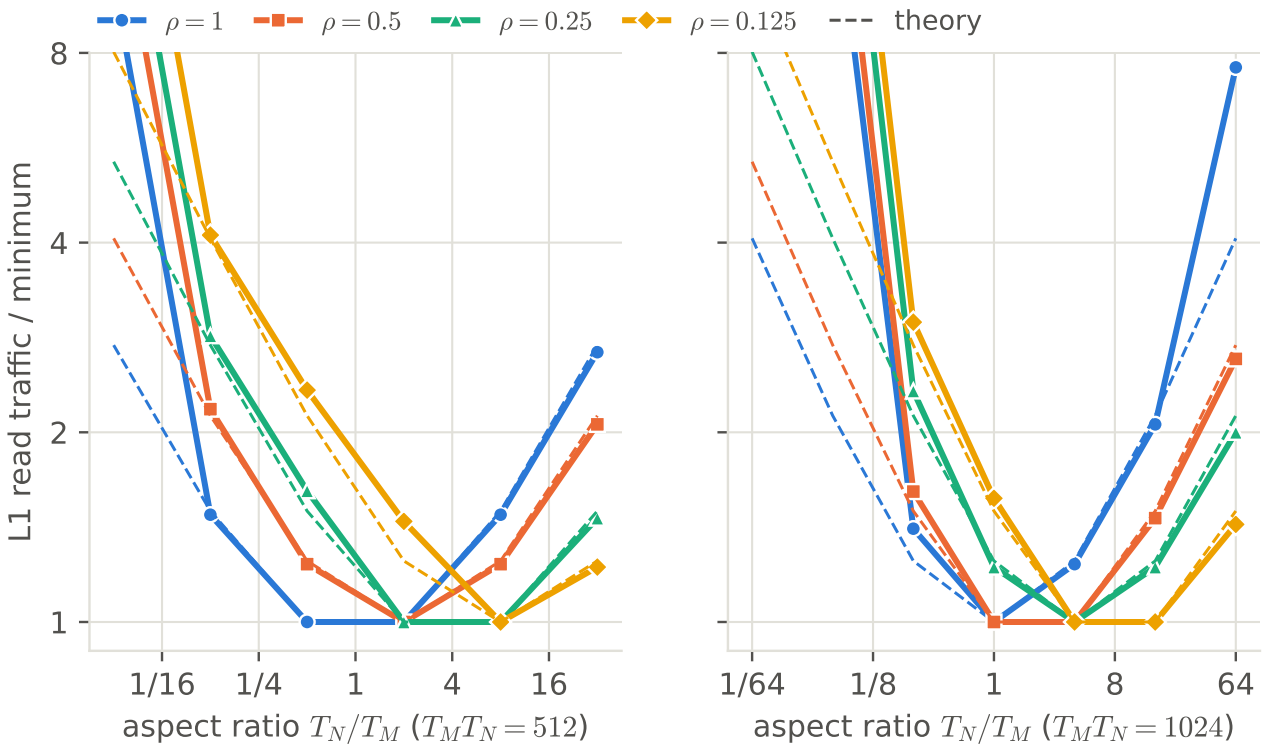
- Tiling keeps tiles in the cache so every loaded element is reused
- All matrices cost the same $\Rightarrow$ square tiles are optimal
- B cheaper to load (a fraction ρ of A) $\Rightarrow$ the tile stretches along B:

$$\frac{T_N}{T_M} = \frac{1}{\rho}$$

Our question: with a PRNG, B costs **time**, not traffic, and it overlaps with the work on A and C. How should we tile now, and how should the hardware be balanced?

Reproducing The Bharadwaj Model

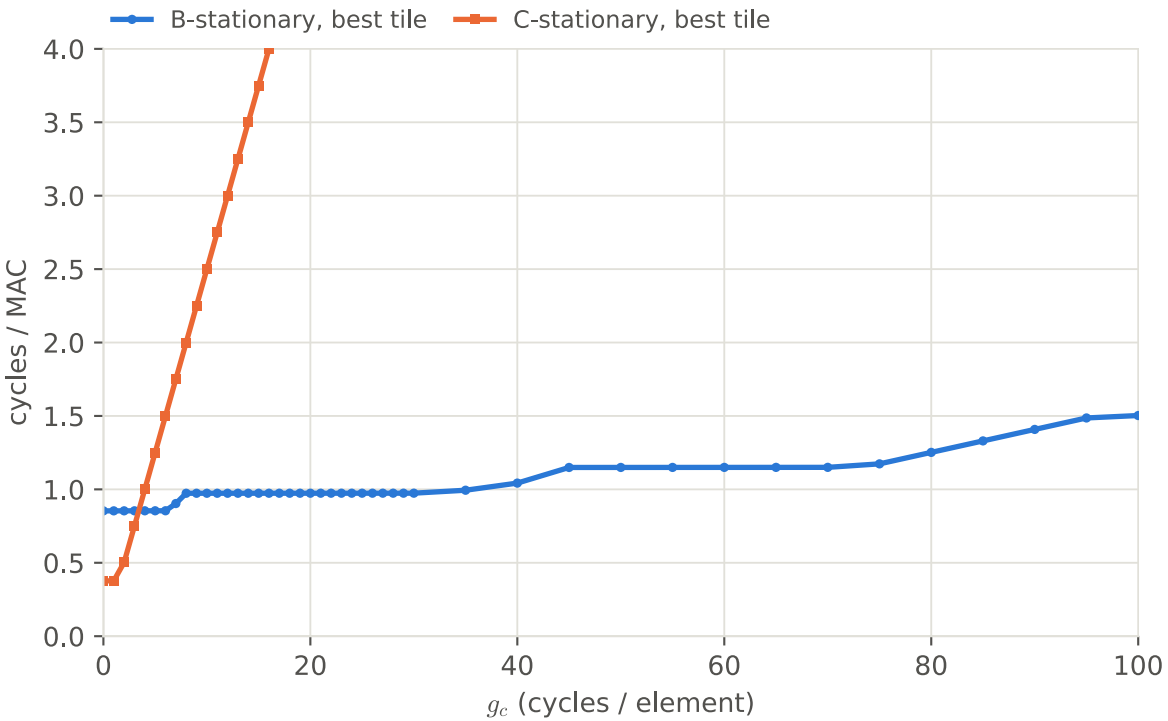
- B stored at lower precision, ratio ρ
- Minimum moves to $T_N/T_M = 1/\rho$
- Savings vs square: 18% and 36% (predicted 20% and 37%)



B-Stationary vs C-Stationary

- Each dataflow at its own best tile, for every g_c
- C-stationary faster only for $g_c \leq 3$
- B-stationary shares each element of B over T_M rows instead of R_M

We use B-stationary



02

The Model

The Bharadwaj Model

- The cost of a tile is its memory traffic per MAC
- An element of A is reused by the T_N columns of the tile, an element of B by its T_M rows
- B costs a fraction ρ of A

$$\frac{\text{traffic}}{\text{MAC}} = \underbrace{\frac{1}{T_N}}_{A} + \underbrace{\frac{\rho}{T_M}}_{B}$$

For a fixed tile area it is smallest at $T_N/T_M = 1/\rho$

Change 1: Traffic To Cycles

- A real cache has hits, misses, tiles that fit or don't
- Too hard to count, so we measure it
- $\alpha(T_M, T_N)$: cycles per MAC for A and C, at $g_c = 0$

Before: $$\frac{1}{T_N} + \frac{\rho}{T_M}$$

After: $$\alpha(T_M, T_N) + \frac{\rho}{T_M}$$

Change 2: Precision To Generation

- B is never stored, it is generated
- g_c cycles per element, shared by the T_M rows of the tile

Before: $$\alpha(T_M, T_N) + \frac{\rho}{T_M}$$

After: $$\alpha(T_M, T_N) + \frac{g_c}{T_M}$$

Change 3: Sum To Max

- The generator runs in the background
- Generation overlaps with the work on A and C
- Only the slower one sets the runtime

$$\text{Before:} \quad \alpha(T_M, T_N) + \frac{g_c}{T_M}$$

$$\text{After:} \quad \max\left(\alpha(T_M, T_N), \frac{g_c}{T_M}\right)$$

Measuring α: Set $g_c = 0$

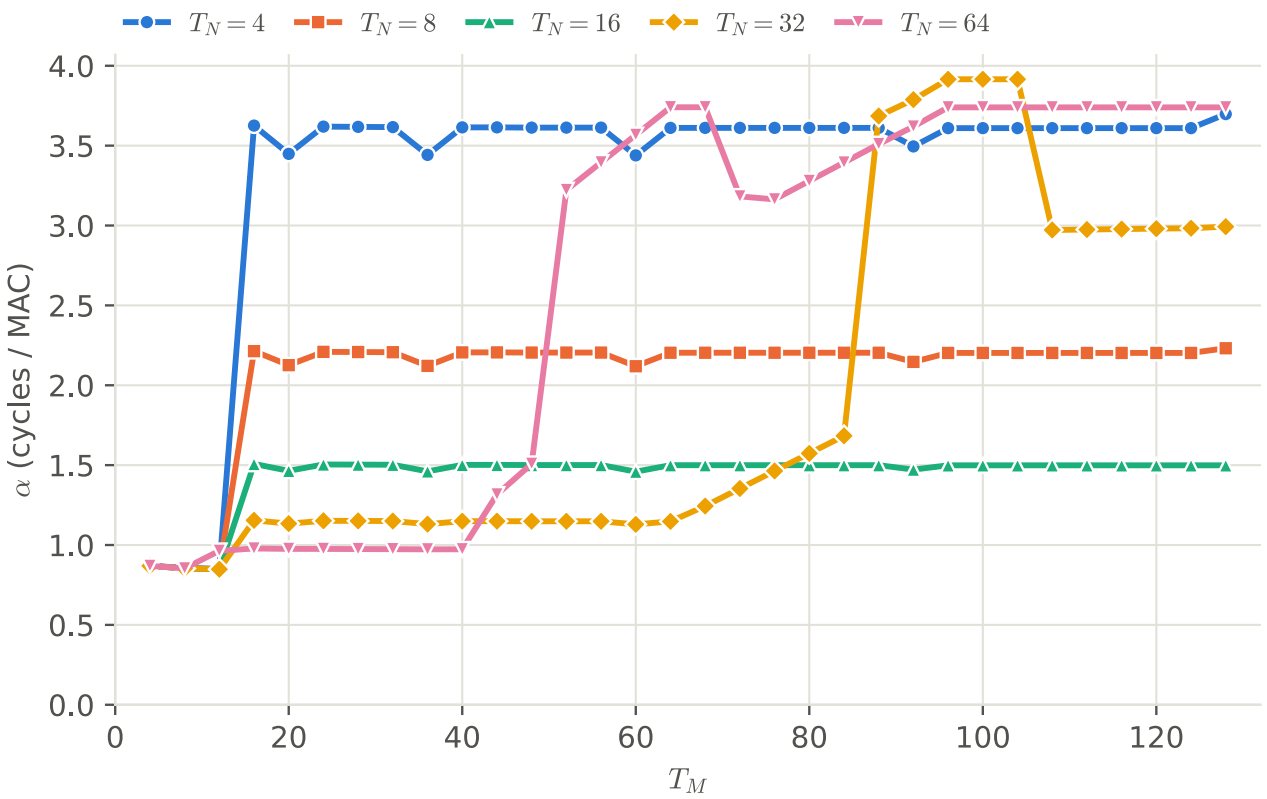
- With $g_c = 0$ the generator is free: B is always waiting in the FIFO
- The generation side of the max disappears
- So the runtime we measure is α itself

$$g_c = 0 \quad \Rightarrow \quad \frac{T}{MNK} = \max\left(\alpha(T_M, T_N), \frac{0}{T_M}\right) = \alpha(T_M, T_N)$$

- One simulation per tile gives a table of α
- The same table then serves every other g_c

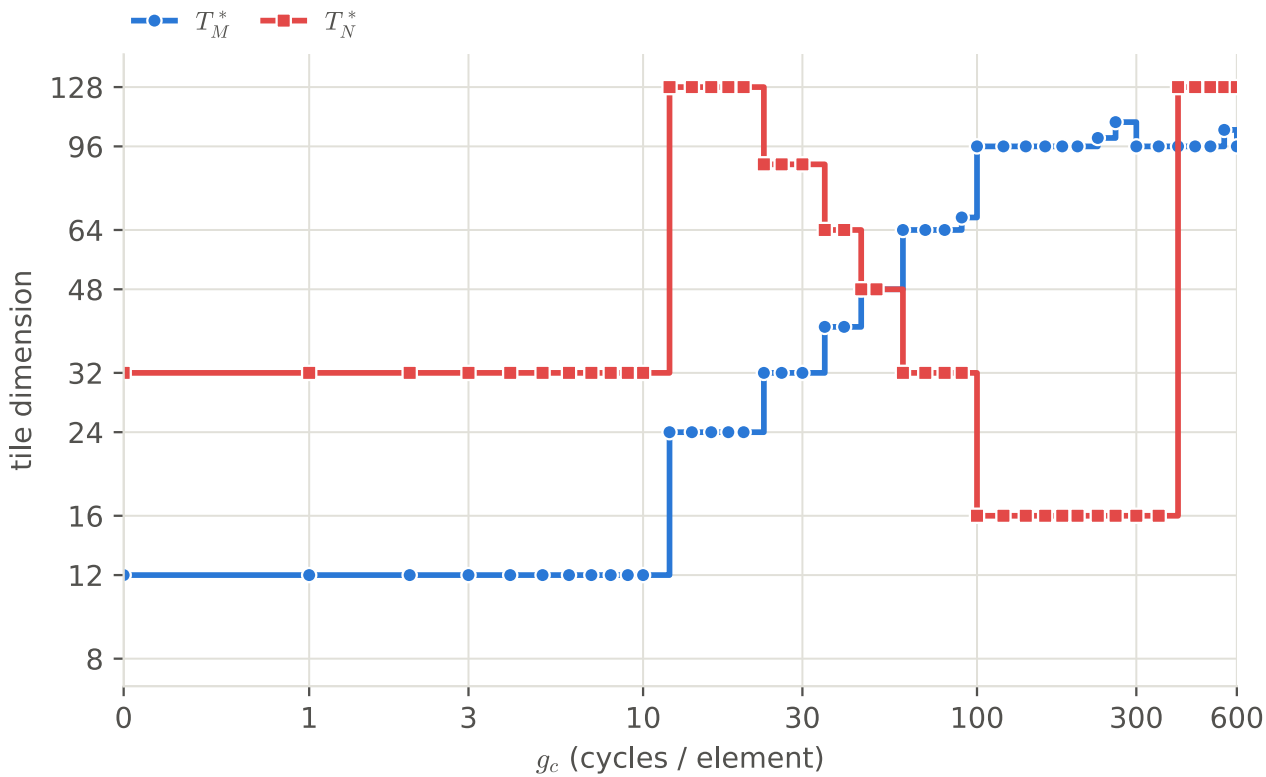
The Memory Cost α

- A step function
- Tile of A fits: cheap
- Strip of A doesn't fit: wider tiles are cheaper
- Tile of C overflows: second cliff



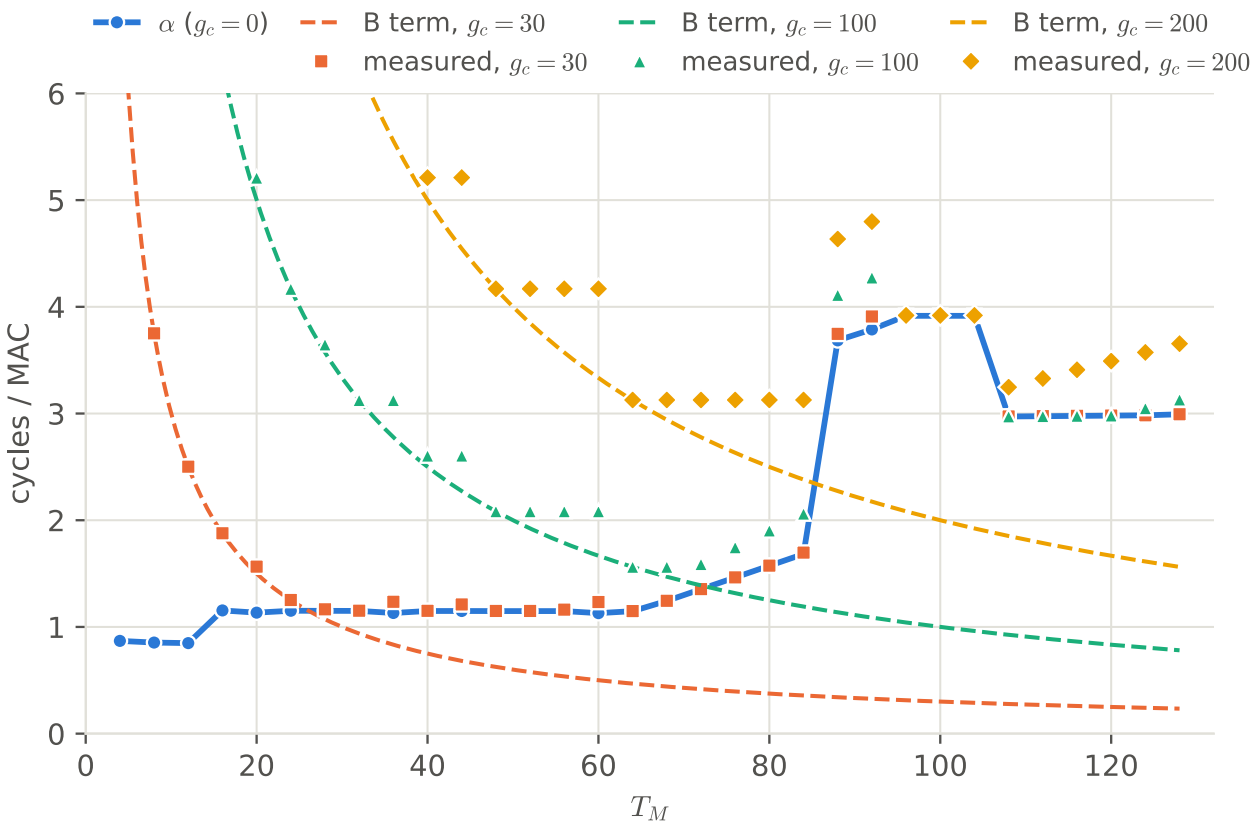
The Best Tile

- T_M grows to hide generation
- T_N grows, then shrinks to delay C tile thrashing
- For a very slow generator $\Rightarrow$ the bigger the better.



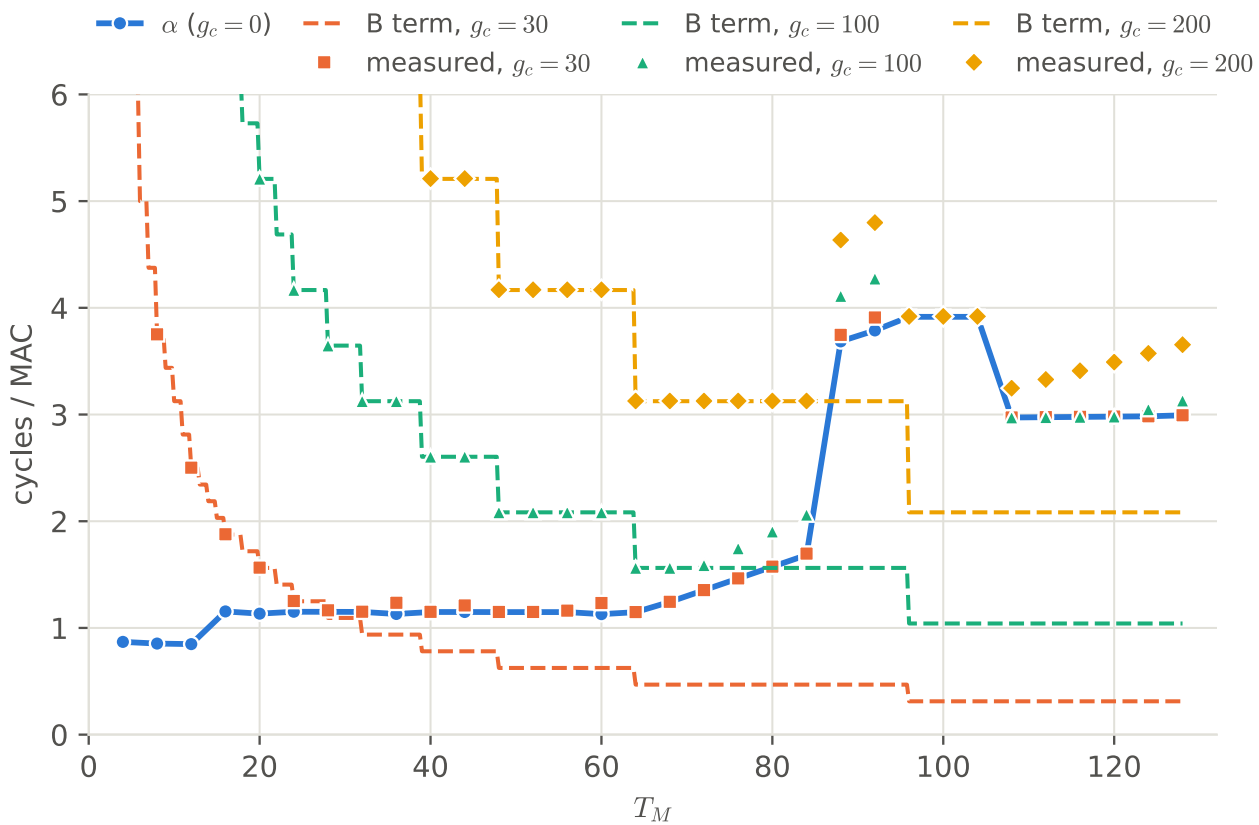
It's a Simple Model

- Runtime follows the higher curve
- Best tile at the crossing
- Misses heights that don't divide M



Refinement 1: B Per Row Of Tiles

- B is generated once per row of tiles
- Generation cost: $g_c \cdot \lceil M/T_M \rceil / M$
- Giving us a staircase



Refinement 2: Startup

- Each tile loads its own seed, the FIFO starts empty
- The core waits for the first block of B: $g_c R_M R_N$ cycles per tile
- Only hurts when memory is the bottleneck $\Rightarrow$ memory side of the max

Before: $$\max\left(\alpha(T_M, T_N), \frac{g_c}{T_M}\right)$$

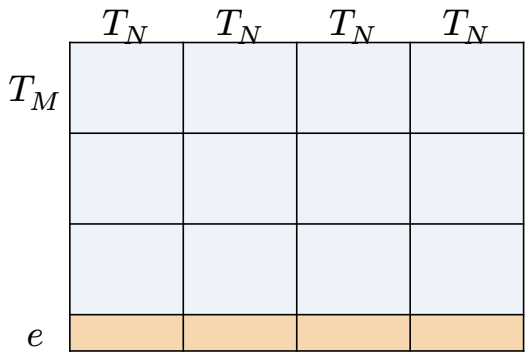
After: $$\max\left(\alpha(T_M, T_N) + \frac{s}{T_M}, \frac{g_c}{T_M}\right)$$

$$s = \frac{g_c R_M R_N \cdot \lceil N/T_N \rceil}{KN}$$

the wait of one tile, times the tiles in a row of tiles, spread over the KN MACs of one row of C

Refinement 3: The Short Last Row (M)

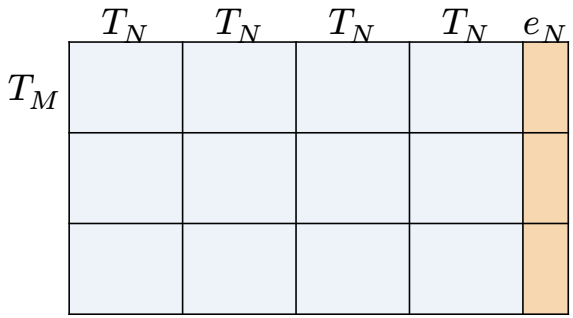
- T_M doesn't divide $M \Rightarrow$ a short last row, height $e = M \bmod T_M$
- Every row of tiles generates all of B
- Full rows wait for memory, the short row waits for generation
- The waits add up $\Rightarrow$ max per row



$$\frac{T}{MNK} = \frac{1}{M}\left(\left\lfloor \frac{M}{T_M} \right\rfloor \cdot \max(T_M \cdot \overline{\alpha}(T_M, T_N) + s, g_c) + \max(e \cdot \overline{\alpha}(e, T_N) + s, g_c)\right)$$

Refinement 3: The Narrow Last Tile (N)

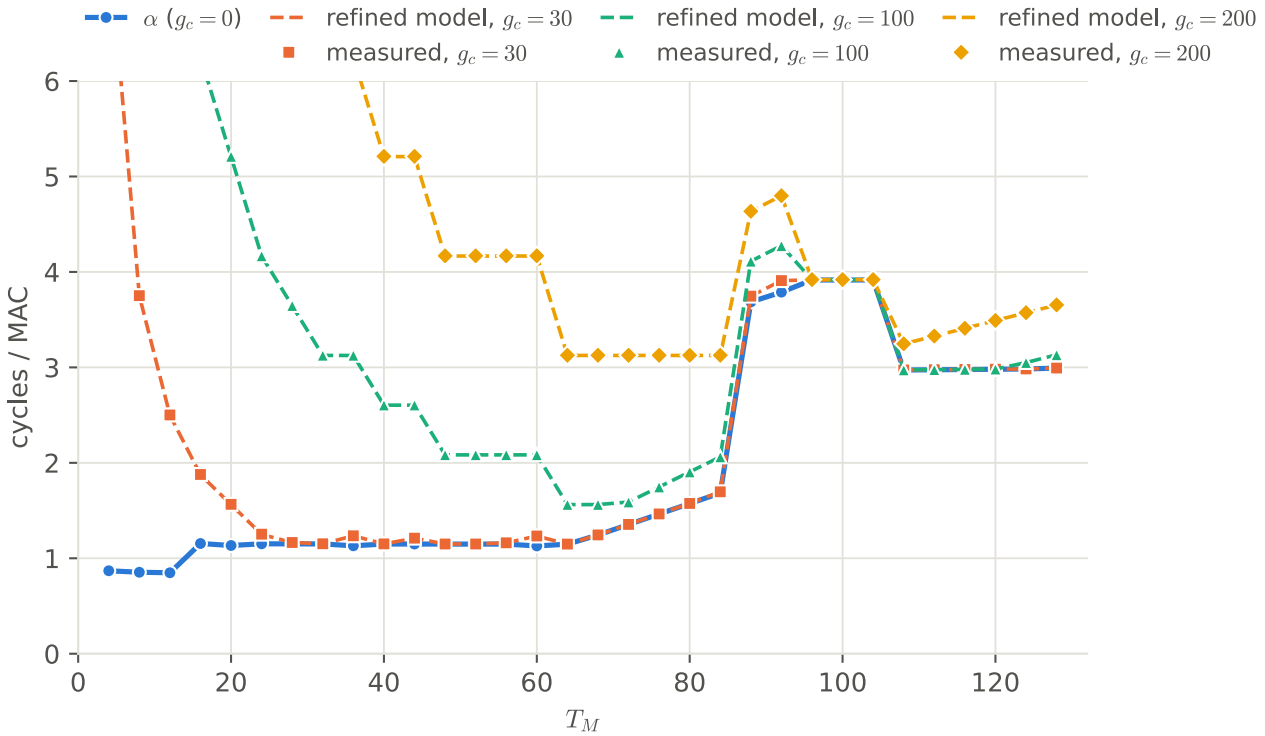
- T_N doesn't divide $N \Rightarrow$ a narrow last tile, width $e_N = N \bmod T_N$
- It has its own α
- $\overline{\alpha}$: the average over the row, weighted by width



$$\overline{\alpha}(r, T_N) = \frac{\left\lfloor \frac{N}{T_N} \right\rfloor \cdot T_N \cdot \alpha(r, T_N) + e_N \cdot \alpha(r, e_N)}{N}$$

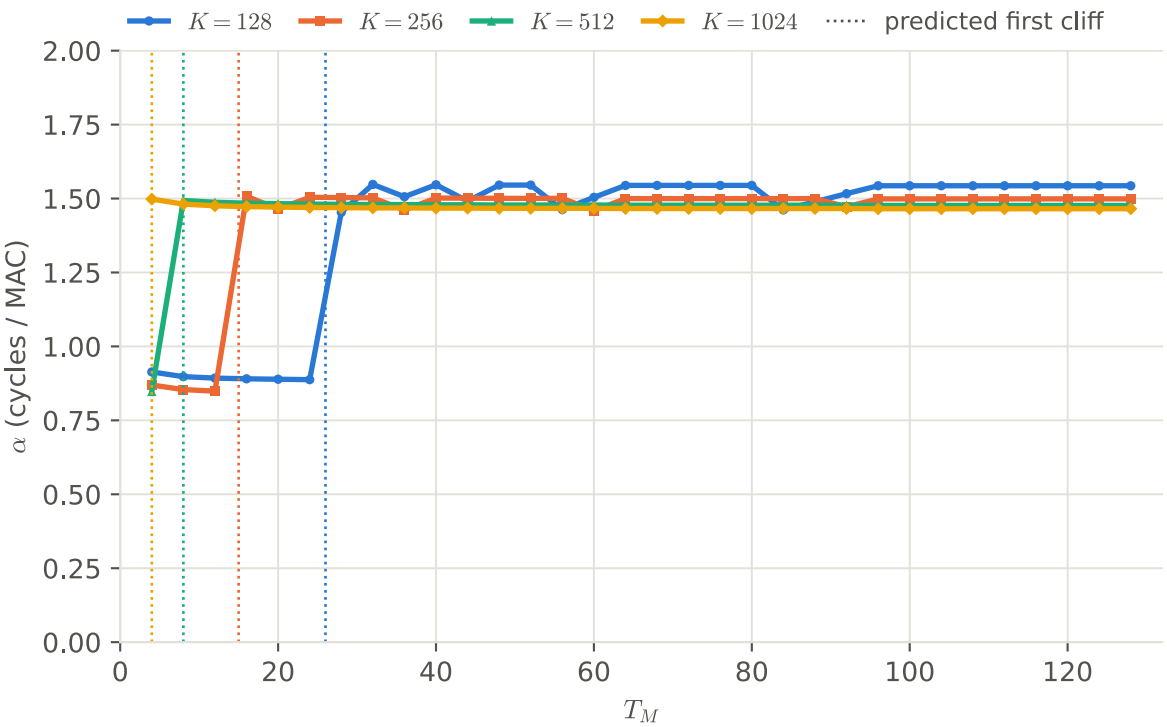
The Refined Model

- Startup on the memory side
- Max per row, short last row separately
- Within 0.2% of the measurements
- Tiles divide the matrix ⇒ back to the simple model



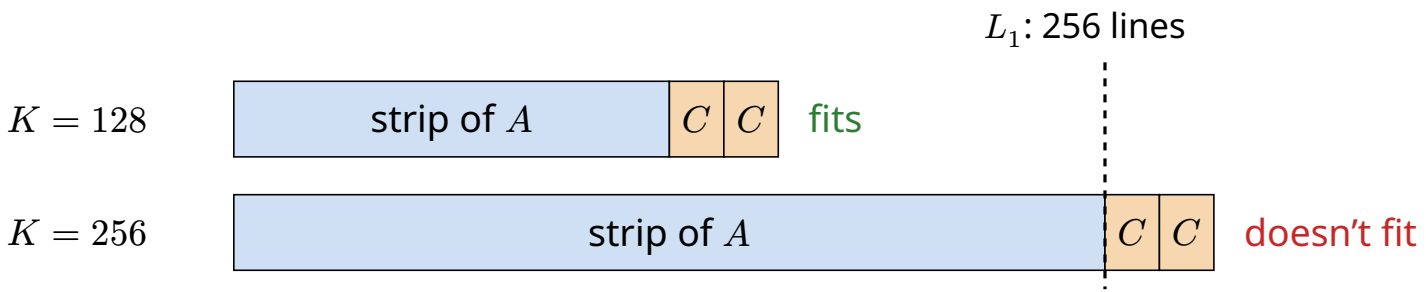
Refinement 4: Generalizing for many matrices

- α was measured at one K
- M and N are in the formula, K isn't
- Every K: the same two levels
- Only the first cliff moves



Refinement 4: Why The Cliff Moves

- A line of A is used in one tile, and again in the next tile
- In between: the rest of the strip of A, this tile of C and the next tile of C
- All of it has to fit in L_1



a row of tiles of height $r = 16$, $T_N = 16$

$$r \cdot \left(\left\lceil \frac{K}{E} \right\rceil + 2 \left\lceil \frac{T_N}{E} \right\rceil \right) < |L_{1 \text{ lines}}|$$

E: elements per cache line (16) $|L_{1 \text{ lines}}|$: lines in L_1 (256)

Refinement 4: A Small Drift In K

- Inside each level, α still drifts a little with K
- The reason is C:
  - every line of C is loaded from memory once, a compulsory miss: L_{mem} cycles for E elements
  - every element of C is used by K MACs

$$\underbrace{\frac{L_{\text{mem}}}{E}}_{\text{per element of } C} \cdot \underbrace{\frac{1}{K}}_{K \text{ MACs share it}} = \frac{L_{\text{mem}}}{EK} \text{ cycles per MAC}$$

L_{mem}: memory latency (180 cycles)

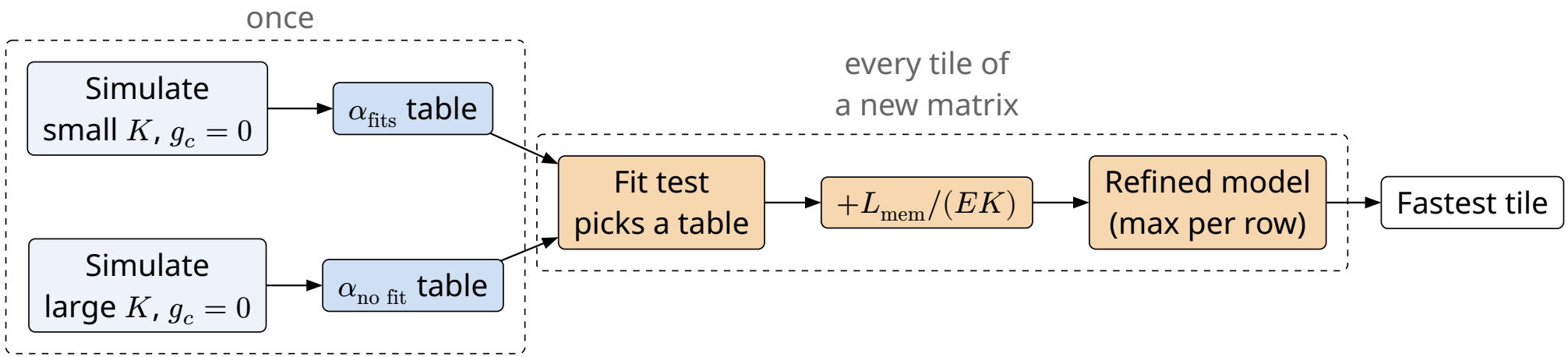
Refinement 4: The Formula

- Two levels $\Rightarrow$ two tables, one calibration each
- Plus the drift in K

Before: $\alpha(r, T_N)$

After: $\begin{cases} \alpha_{\text{fits}}(r, T_N) & \text{if the row fits} \\ \alpha_{\text{no fit}}(r, T_N) & \text{otherwise} \end{cases} + \frac{L_{\text{mem}}}{EK}$

Refinement 4: The Whole Process



03

Software Focused Design Questions

QUESTION 1

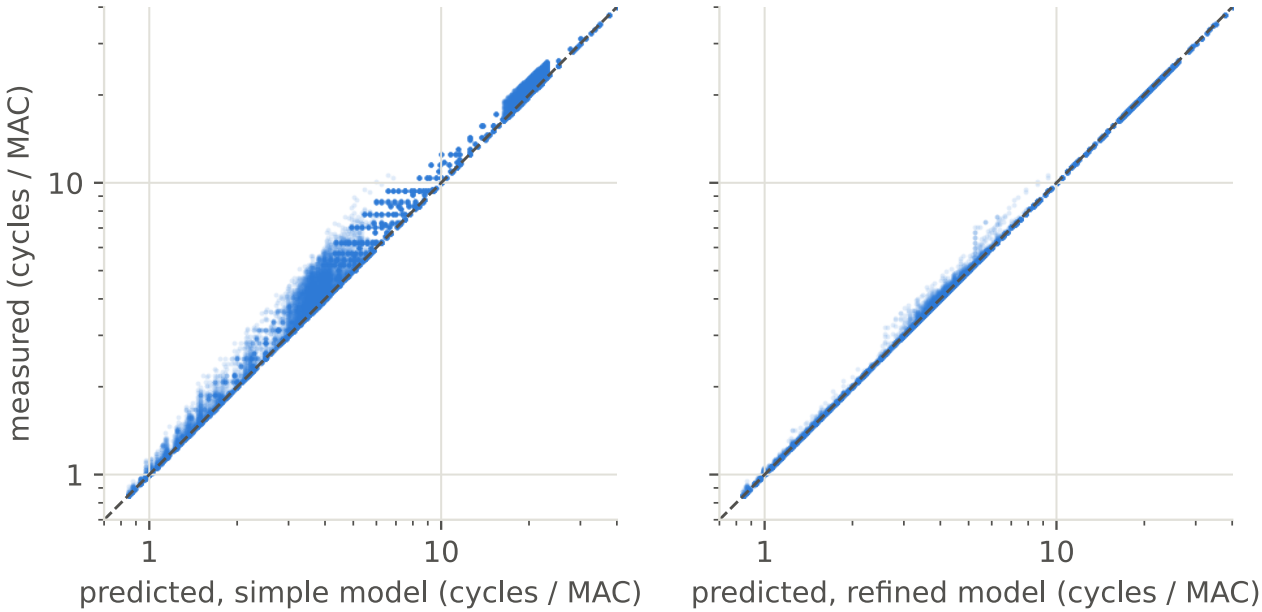
Can Our Model Be Trusted To Choose The Tile?

Can The Model Choose The Tile?

1. Calibrate α once at: $g_c = 0$
2. Predict every tile
3. Pick the fastest

results:

- Within 1% in 96% of runs
- Picked tile at most 0.1% slower

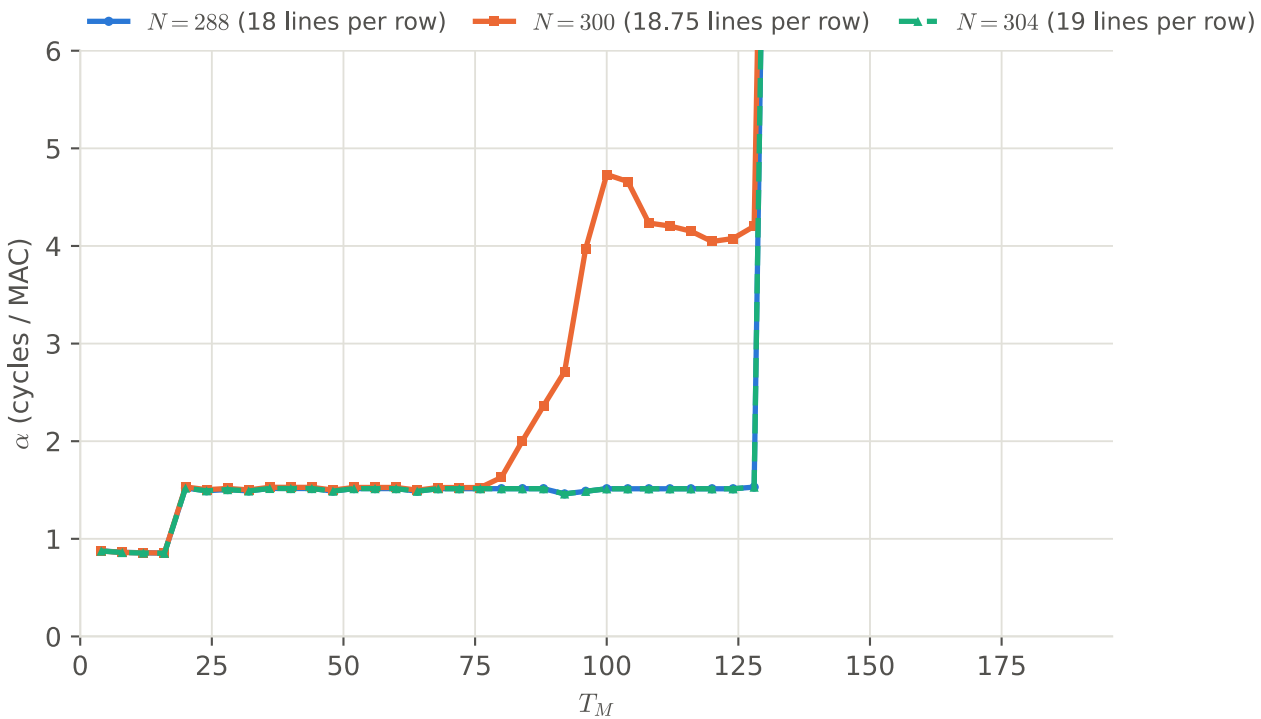


QUESTION 2

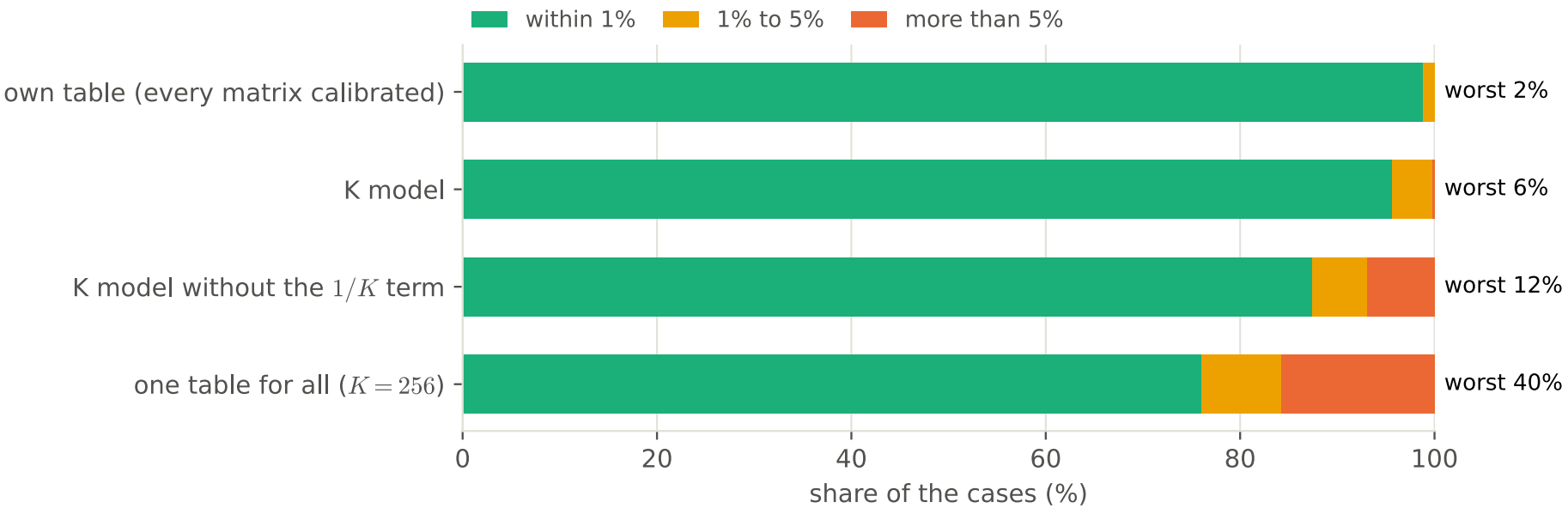
Does The Model Work For Other Matrices?

Other Matrices: Pad The Rows

- Rows of C that not whole cache lines cause earlier thrashing
- Solution: Pad every row to whole lines



Other Matrices: Two Calibrations



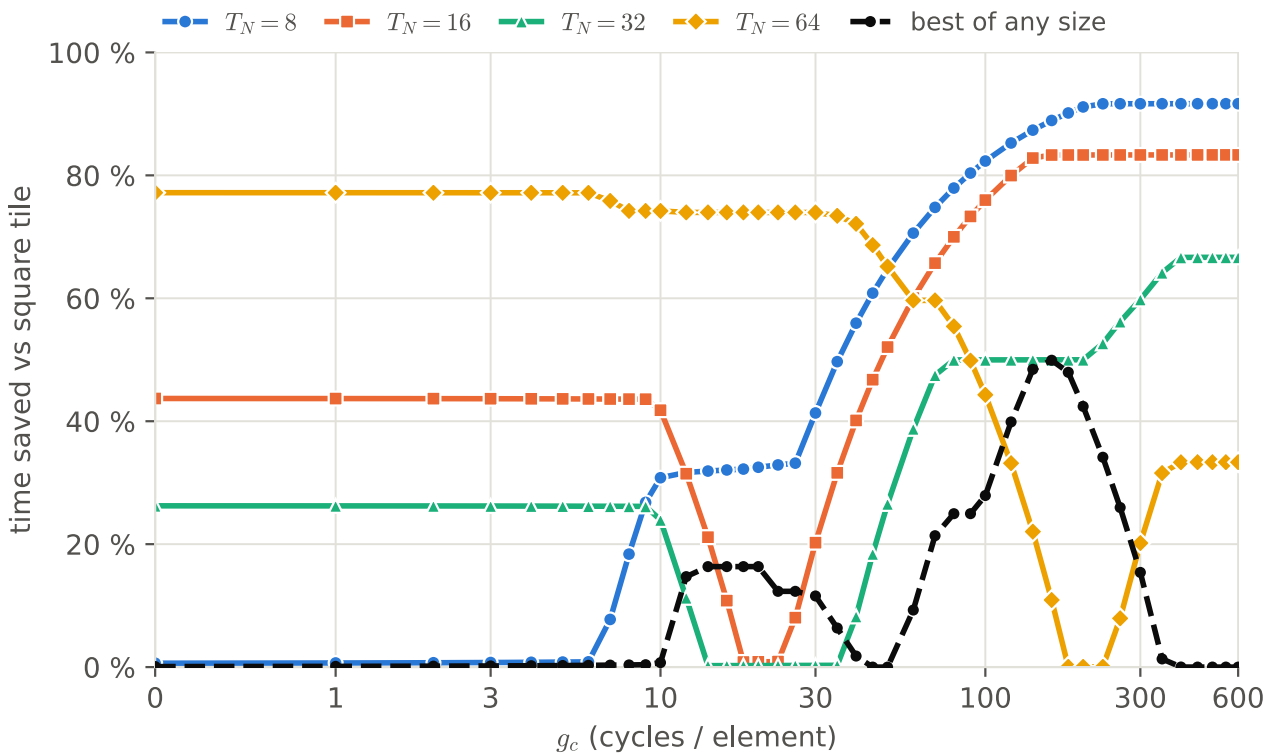
- Within 1% in 96%
- Works for K outside the calibrations

QUESTION 3

Does The Choice Of Tile Matter?

Does The Choice Matter?

- Never worse than a square
- Saves up to half of the runtime
- Even when a square is best, the model tells which square



04

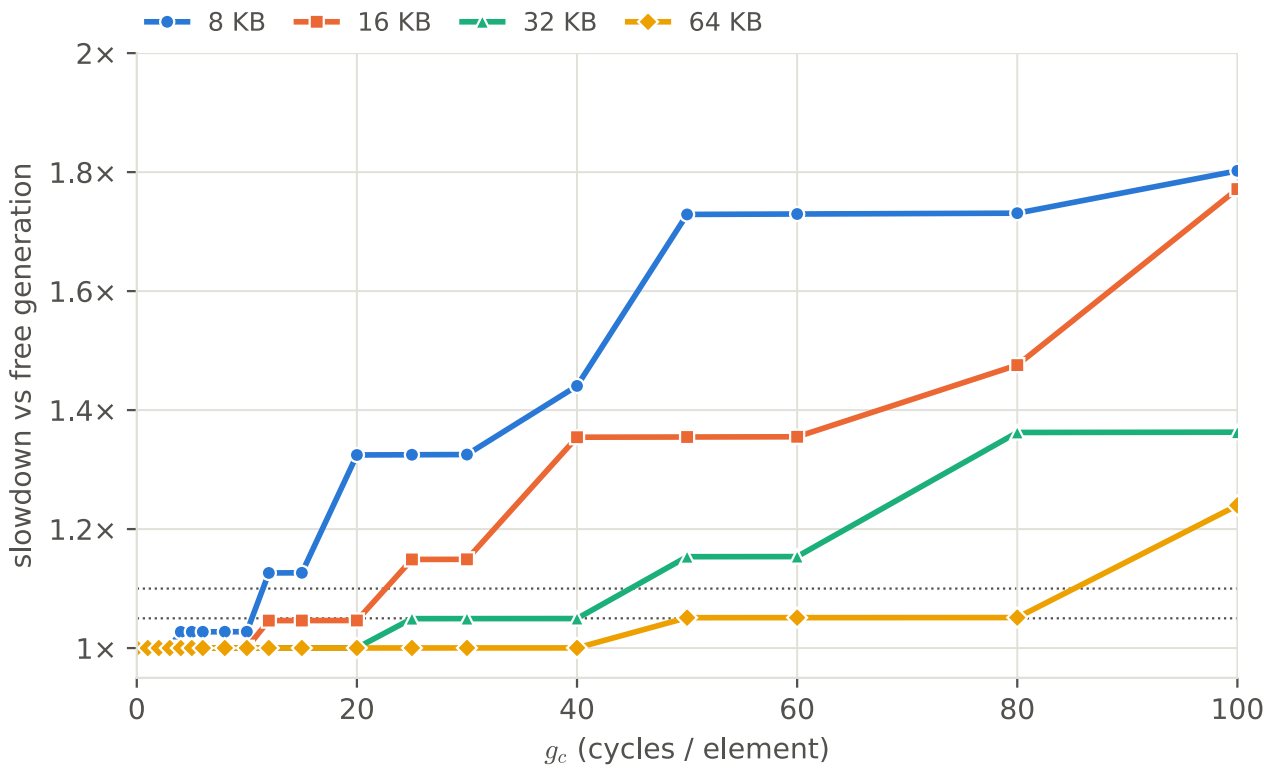
Hardware Focused Design Questions

QUESTION 1

How Fast Must The Generator Be?

How Fast Must The Generator Be?

- Generation is as good as free up to $g_c \approx T_M \cdot \alpha$
- Doubling the cache $\approx$ halving g_c

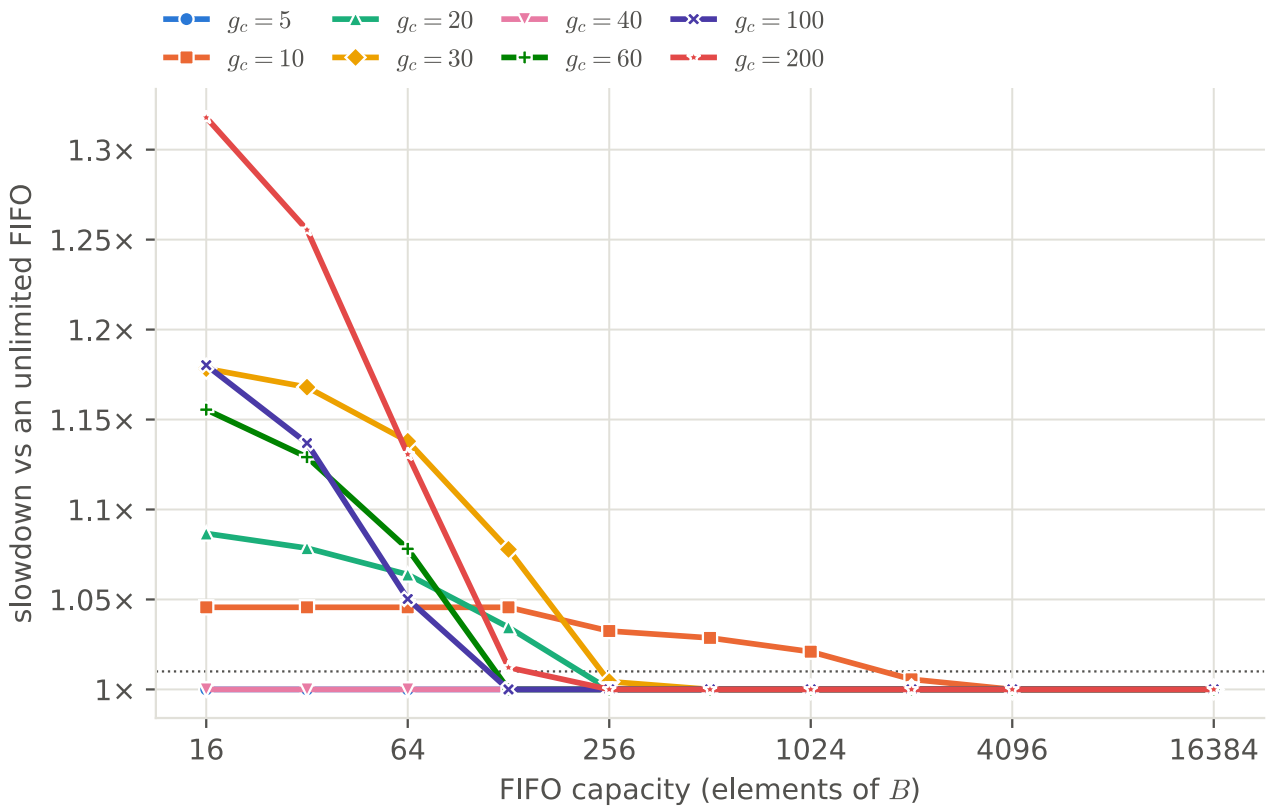


QUESTION 2

How Big Must The FIFO Be?

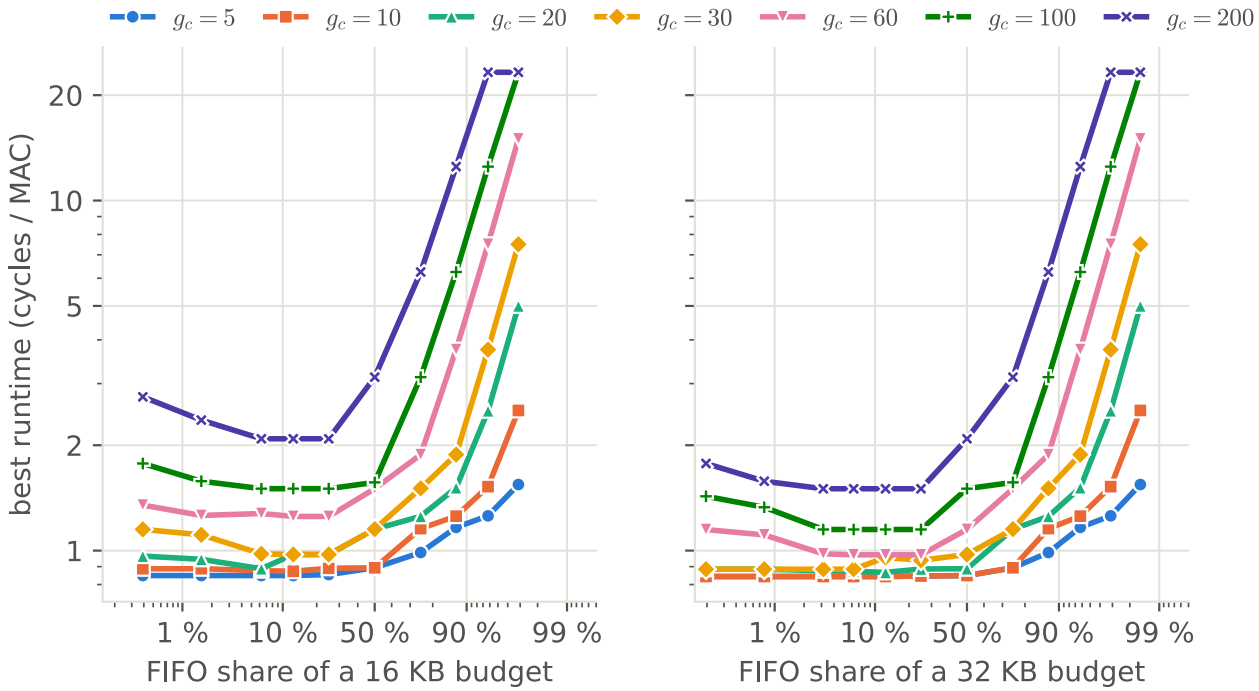
How Big Must The FIFO Be?

- Less slowdown as capacity grows
- Slower generation can sometimes afford a smaller FIFO



Bigger Cache Or Bigger FIFO?

- When too much goes to the cache ⇒ slightly slower runtime
- When too much goes to the FIFO ⇒ horrible performance
- As soon as the FIFO is big enough the rest should go to the cache

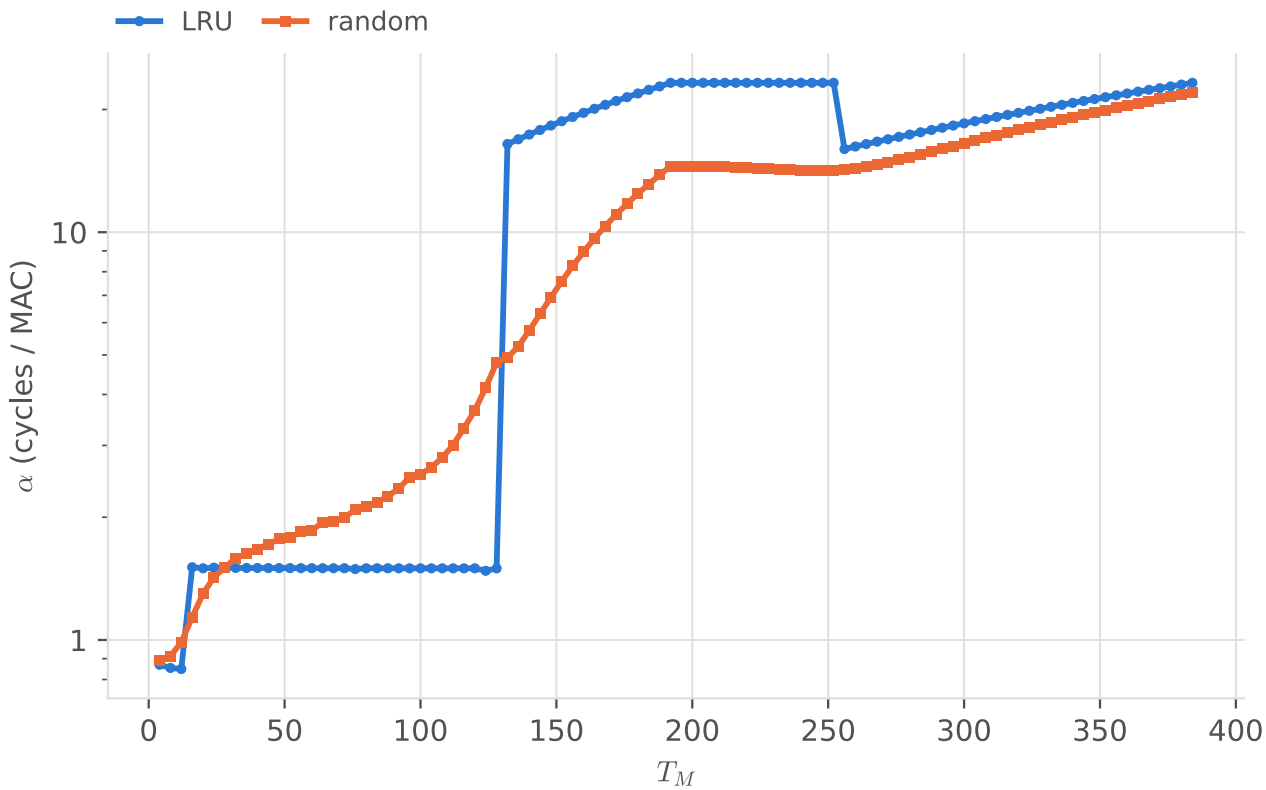


QUESTION 3

Does The Replacement Policy Matter?

Does The Replacement Policy Matter?

- Random smooths the cliffs
- LRU evicts the line needed next in a loop
- Random lets lines survive by chance



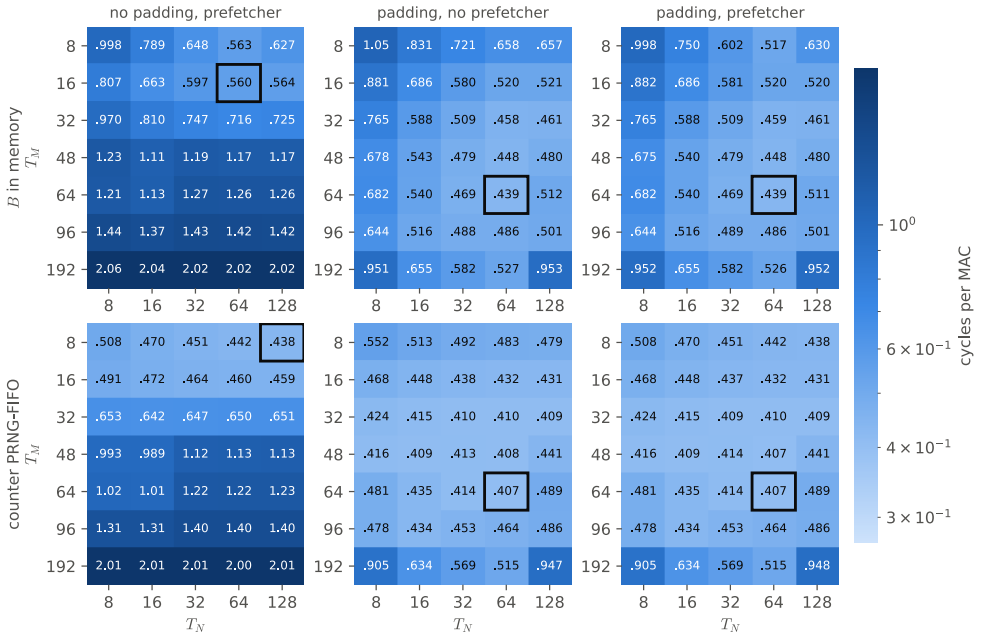
05

gem5 Simulation

gem5 Setup

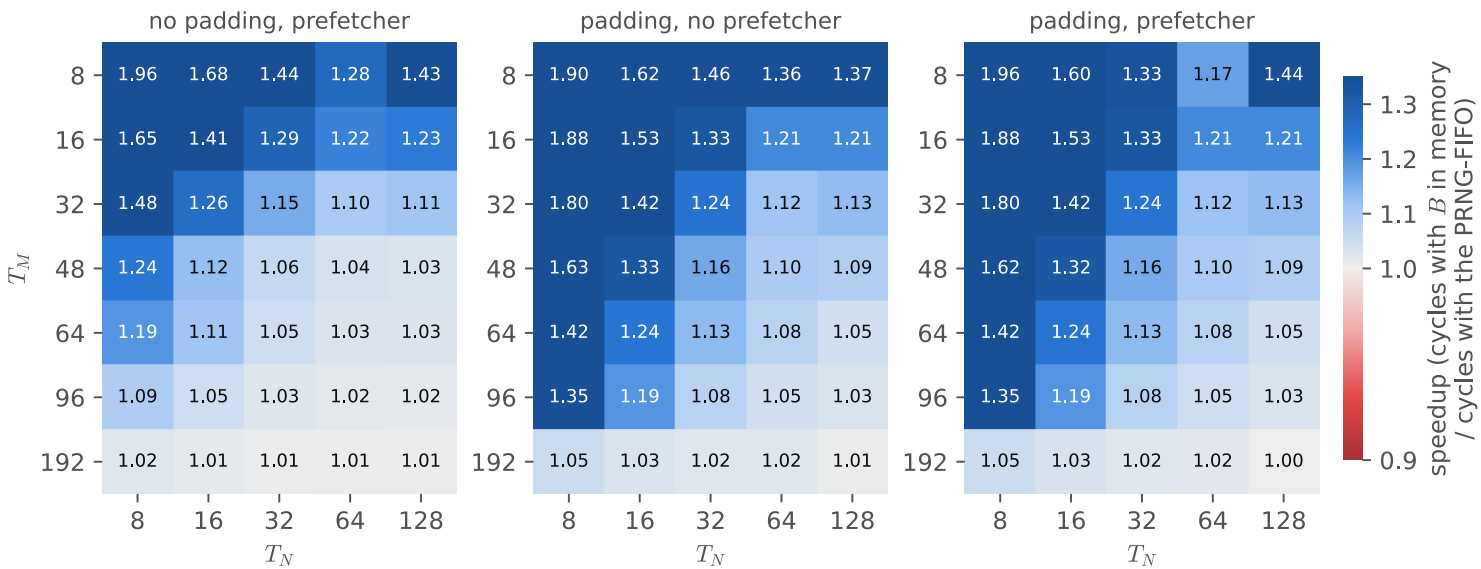
- Our simulator charges every miss in full. On a real core misses overlap
- gem5: the sizes of TI's C7x DSP (C7524), 1 GHz, 32 KB L_1D, 64 KB L_2, 256-bit vectors
- gem5's in-order Arm core (`MinorCPU`) with SVE
- Two sources of B (int8): B **in memory**, or the **PRNG-FIFO** between the core and L_1
- $192 \times 128 \times 512$, 35 tiles, $g_c = 0$ first, then 1 to 128

gem5: Cycles Per MAC ($g_c = 0$)



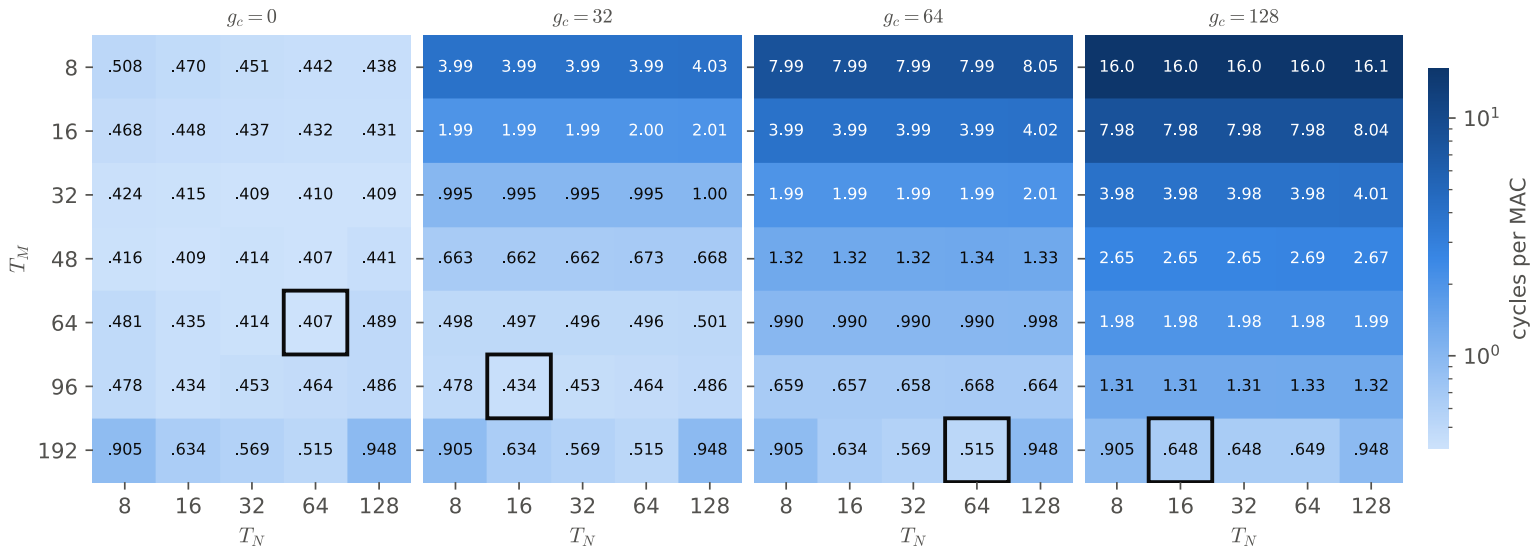
- With padding: best tile 64×64 for both, 0.44 (B in memory) and 0.41 (PRNG-FIFO) cycles per MAC
- Without padding: set conflicts rule, only short tiles are good

gem5: Speedup Of The PRNG-FIFO



- Same tile: 1.00× to 1.96×, it follows the traffic of B ($\propto 1/T_M$)
- Best tile against best tile: 8%, and 28% without padding

gem5: A Slow Generator



- Up to $g_c = 16$ nothing changes: generation is hidden
- From $g_c = 32$ the best tile grows taller: 96×16, then 192×64, then 192×16

gem5: Branch Prediction And The FIFO

- The core runs ahead after a mispredicted loop exit
- A load of B on that wrong path reaches the FIFO before the branch resolves
- Counter PRNG-FIFO: the extra elements are thrown away, costs only generation time
- Seeded PRNG-FIFO: the stream shifts, the rest of the tile gets the **wrong** B

```
tile 1: seed1 -> b1 b2 b3 b4 b5 ...
  needs:  step 1: b1 b2   step 2: b3 b4
  gets:   step 1: b1 b2   (b3 on the wrong path)   step 2: b4 b5   <- wrong C
tile 2: seed2 -> FIFO emptied and restarted, the shift does not carry over
tile 3: seed3 -> ...
```

> **FIFO must pop only on reads that commit**

Future Studies

- More matrices and cache sizes, to see how the thresholds move
- Going in much further on every one of the experiments we showed
- Researching the mathematical model further to achieve a deterministic model
- Predict α from the cache parameters instead of calibrating it for every new hardware

Thank You